

따라하며 끝내는 시리즈

따라하며 끝내는

AICE Associate

초보자를 위한 설치부터 합격까지 - 이론과 실무를 한 번에

실제 문서 사이트 화면 구성 수록 · 60 개+ 실전 예제 · 노트북 39 개 연동

목차 (Table of Contents)

들어가며 - AICE Associate, 왜 지금 시작해야 할까요

PART 1. AICE Associate 완전 이해하기

- 1 장. AICE Associate 란 무엇인가
- 2 장. 왜 지금 AICE Associate 를 취득해야 하는가
- 3 장. 시험 완전 분석 - 구조, 시간, 유형, 배점
- 4 장. 나는 파이썬을 얼마나 알아야 할까?

PART 2. 시작하기 전 준비 (실전 화면 수록)

- 5 장. 개발 환경 설치 - 내 컴퓨터에 실습 환경 만들기
- 6 장. 프로젝트 폴더 구조 사용법

PART 3. 이론 완전 정리 (초보자를 위한 쉬운 요약)

- 7 장. 데이터 분석의 기초 - EDA
- 8 장. 데이터 전처리 완전 정리
- 9 장. 데이터 시각화 완전 정리
- 10 장. 머신러닝 - 회귀 완전 정리
- 11 장. 머신러닝 - 분류(이진/다중) 완전 정리
- 12 장. 머신러닝 심화 - 교차검증·튜닝·파이프라인
- 13 장. 딥러닝 완전 정리

PART 4. 실무 감각 익히기

- 14 장. 실전 코드 템플릿 총정리
- 15 장. 자주 발생하는 에러와 해결법
- 16 장. 실제 문제풀이 예시 (실행 화면 수록)

PART 5. 합격 전략

- 17 장. 오픈북 시험 전략 8 가지 + α
- 18 장. 시간 배분 전략 (90 분 분배)
- 19 장. 자주 하는 실수 Top 10

20 장. 14 일 학습 로드맵

마치며 - AICE Associate 합격, 이제 시작입니다

부록 A. 문제유형별 모델·모듈·평가함수 총정리표

부록 B. 폴더/노트북 전체 목록

부록 C. 추가 실전 예시 (실행 화면 수록)

부록 D. 핵심 용어 사전

부록 E. 자주 묻는 질문 (FAQ)

부록 F. 학습 체크리스트

부록 G. 딥러닝 모듈 경로 총정리

부록 H. 찾아보기 - 핵심 함수 색인

"파이썬을 하나도 몰라도, 이 책을 따라만 오시면 됩니다."

들어가며

코딩 학원을 다니다 보면 누구나 한 번쯤 '자격증 하나쯤은 있어야 하지 않을까'라는 생각을 하게 됩니다. 그런데 정작 자격증 준비를 시작하려고 하면 막막합니다. 어디서부터 시작해야 할지, 뭘 설치해야 할지, 이론을 얼마나 깊게 알아야 하는지 감이 오지 않기 때문입니다.

이 가이드는 그 막막함을 없애기 위해 만들어졌습니다. **AICE Associate**는 KT가 개발하고 한국경제신문이 주관하는 국내 최초의 실무형 AI 자격증으로, 2024년 12월부터 AI 분야 최초의 국가공인자격이 되었습니다. '인공지능', '머신러닝', '딥러닝'이라는 단어만 들으면 어렵게 느껴지지만, 이 시험이 실제로 요구하는 것은 생각보다 단순합니다. **데이터를 정리하고, 모델을 만들어 학습시키고, 예측과 평가를 하는 정해진 흐름**을 코드로 옮길 수 있으면 됩니다.

많은 사람들이 '오픈북이니까 검색하면서 풀면 되지 않을까?'라고 생각합니다. 하지만 2025년부터 오픈북 기준이 '자유 검색'에서 '**지정된 공식 문서 7종만 열람 가능**'으로 바뀌면서, 사전 지식 없이 시험장에서 처음 검색하며 문제를 푸는 전략은 통하지 않게 되었습니다. 기초 개념을 미리 이해하고 있어야, 공식 문서에서 원하는 함수를 빠르게 찾아 쓸 수 있습니다.

이 프로젝트는 그래서 두 가지를 동시에 준비했습니다. 하나는 지금 읽고 계신 **이 가이드**이고, 다른 하나는 실제로 손으로 풀어보는 **39개의 Jupyter Notebook 실습 파일**입니다. 가이드에서 '왜'를 이해하고, 노트북에서 '어떻게'를 손에 익히는 구조입니다.

이 가이드만의 차별점

시중의 다른 대비서와 달리, 이 가이드는 **오픈북 허용 공식 문서 7종의 실제 화면 구성**을 그대로 재현해 수록했습니다. 시험장에서 마주할 화면을 미리 눈에 익혀두면, 실전에서 '이 화면 어디서 봤는데...' 하며 헤매는 시간을 크게 줄일 수 있습니다. 또한 39개의 연동 노트북이 함께 제공되어, 가이드에서 배운 개념을 바로 그 자리에서 코드로 확인할 수 있습니다.

혹시 지금 '나는 코딩을 거의 안 해봤는데 괜찮을까?' 하는 걱정이 든다면, 그 걱정은 잠시 내려두셔도 됩니다. 이 가이드와 노트북들은 **Python 기초 문법조차 확신이 없는 분들**을 기준으로 설계되었습니다. 필요한 모든 코드는 이미 이 시리즈 어딘가에 예제로 준비되어 있고, 여러분이 할 일은 그 코드를 이해하고 비슷한 문제에 응용하는 연습을 반복하는 것뿐입니다.

준비되었다면, 다음 장부터 차근차근 시작해봅시다.

PART 1

AICE Associate 완전 이해하기

시험이 무엇을 평가하는지, 왜 지금 준비해야 하는지,
얼마나 알아야 시작할 수 있는지부터 확실히 잡습니다

1 장. AICE Associate 란 무엇인가

AICE(AI Certificate for Everyone)는 KT가 개발하고 한국경제신문이 주관하는 AI 활용 능력 평가 시험입니다. 등급은 여러 단계로 나뉘어 있으며, 이 중 **Associate** 등급은 표(Tabular) 형태의 데이터를 Python으로 분석하고 모델링하는 실무 능력을 검증합니다.

항목	내용
주관	KT 개발 / 한국경제신문 주관
등급	Associate (Tabular 데이터 분석·모델링)
공인 여부	2024.12~ AI 분야 최초 국가공인자격
평가 대상	데이터 분석, 전처리, 머신러닝/딥러닝 모델링 실무 능력
시험 시간	90 분
합격 기준	100 점 만점 중 80 점 이상
시험 방식	온라인 실기(코딩) 시험, 오픈북(단 지정 문서 7 종만 열람 가능)

가장 먼저 기억해야 할 것은, 이 시험이 **딥러닝의 수학적 원리(역전파, 경사하강법의 미분식 등)**를 묻는 시험이 아니라는 점입니다. '이 문제 상황에서 어떤 함수를 어떤 옵션으로 써야 하는가'를 빠르고 정확하게 아는지를 검증하는 **실무형 시험**입니다. 대학원 입시나 이론 자격증과는 성격이 완전히 다릅니다.

그렇다고 이론이 필요 없다는 뜻은 아닙니다. 함수 사용법만 기계적으로 외우면 풀리는 문제도 있지만, 문제의 표현이 조금만 바뀌어도(예: '평균으로 채우세요' → '중앙값으로 채우세요') 당황하지 않으려면 **왜 이 상황에서 이 함수를 쓰는지**를 이해하고 있어야 합니다. 이 가이드와 연동된 노트북 시리즈 전체가 '함수 암기'가 아니라 '왜 이 단계에서 이 함수를 쓰는가'에 집중하는 이유가 여기에 있습니다.

2 장. 왜 지금 AICE Associate 를 취득해야 하는가

AICE Associate 는 '이론 자격증'이 아니라 '실무 도구 사용 자격증'에 가깝습니다. 데이터 수집 → 탐색적 데이터 분석(EDA) → 전처리 → 모델링 → 평가라는, 실제 회사에서 데이터 분석·AI 업무를 할 때마다 반복하는 파이프라인을 손으로 직접 짜보게 만듭니다.

2-1. 국비지원 과정 이수자에게 특히 의미가 큰 이유

국비지원 부트캠프 등에서 머신러닝·딥러닝 이론을 배웠다면, 그 지식을 검증하고 '자격증'이라는 형태로 증명할 수 있는 가장 직접적인 방법이 AICE Associate 입니다. 강의에서 배운 pandas·scikit-learn·TensorFlow 코드가 실제로 손에 익었는지 확인하는 셈이며, 이 과정 자체가 이론을 완전히 내 것으로 만드는 가장 확실한 복습이 됩니다.

2-2. 국가공인자격이라는 무게감

2024 년 12 월부터 AICE 는 AI 분야 최초의 국가공인자격이 되었습니다. 민간 자격증이 난립하는 상황에서, 국가가 공인한 자격증은 이력서와 포트폴리오에서 신뢰도 있는 근거로 작용합니다. 특히 데이터 관련 직무로 취업이나 이직을 준비 중이라면, '실제로 pandas/scikit-learn/TensorFlow 로 문제를 풀어낼 수 있다'는 것을 증명하는 근거 자료가 됩니다.

2-3. 준비 과정 자체가 실무 훈련이 된다

이 자격증을 준비하는 과정 자체가 실무 데이터를 다루는 훈련이 됩니다. 결측치를 어떻게 처리할지, 범주형 변수를 어떻게 인코딩할지, 어떤 모델을 언제 쓸지 - 이 모든 판단은 실제 데이터 분석 업무에서도 매일 반복되는 의사결정입니다. 시험 준비를 '시험만을 위한 공부'가 아니라 '실무 역량을 기르는 과정'으로 바라보면 학습 동기가 훨씬 오래갑니다.

이 가이드의 목표

단순히 '합격'이 아니라, 문제를 몸으로 기억할 만큼 반복해서 풀어보고, 왜 그렇게 푸는지 남에게 설명할 수 있는 수준이 되는 것입니다. 이 정도 수준에 도달하면 시험 합격은 자연스러운 결과물이 됩니다.

3 장. 시험 완전 분석 - 구조, 시간, 유형, 배점

3-1. 시험 진행 방식

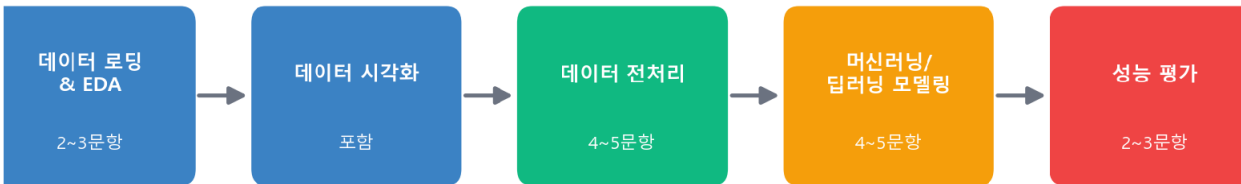
AICE Associate 는 온/오프라인으로 진행되는 **실기(코딩) 시험**입니다. 정해진 시험 시간에 로그인하면 문제지와 함께 코딩 환경(주로 Jupyter Notebook 기반)이 주어지고, 문제에서 요구하는 코드를 작성해 제출합니다. 시험 시간은 **90 분**이며, 100 점 만점에 **80 점 이상**이면 합격입니다.

3-2. 출제 범위 - 큰 흐름은 항상 동일

실제 문항 수와 배점은 회차마다 조금씩 달라질 수 있지만, 아래의 큰 흐름은 거의 항상 동일하게 유지됩니다. 시험지를 처음 받았을 때 이 흐름만 기억하고 있어도 당황하지 않을 수 있습니다.

구분	문항 수(대략)	내용
데이터 로딩 & EDA	2~3 문항	라이브러리 불러오기, 데이터 확인, 기초 시각화
데이터 전처리	4~5 문항	결측치·이상치·인코딩·스케일링·데이터 분할
머신러닝/딥러닝 모델링	4~5 문항	모델 생성·학습·예측
성능 평가	2~3 문항	평가지표 계산, 결과 시각화, 해석

AICE Associate 출제 흐름 (총 90분)



※ 문항 수는 회차마다 조금씩 달라질 수 있지만, 이 순서(흐름)는 항상 동일합니다.

3-3. 오픈북 규칙 - '자유 검색'이 아니라 '제한형 오픈북'

가장 중요하게 짚고 넘어가야 할 부분입니다. 많은 응시자가 '오픈북이니 검색하면서 풀면 된다'고 생각하지만, AICE Associate 는 **지정된 공식 문서 7 종만 열람 가능한 제한형 오픈북**입니다.

구글 검색, 블로그, ChatGPT 같은 생성형 AI 도구는 사용할 수 없습니다.

라이브러리	공식 문서 주소 (클릭 시 이동)
numpy	numpy.org/doc/stable/reference/
pandas	pandas.pydata.org/docs/reference/index.html
matplotlib	matplotlib.org/stable/api/index.html
seaborn	seaborn.pydata.org/api.html
scikit-learn	scikit-learn.org/stable/api/index.html
tensorflow(keras)	tensorflow.org/api_docs/python/tf/keras
XGBoost	xgboost.readthedocs.io/en/stable/python/python_api.html

공식 문서는 기능 설명은 정확하지만 친절한 예제가 부족한 경우가 많습니다.

그래서 **기초 개념과 함수 이름을 미리 알고 있어야**, 시험 중 공식 문서에서 원하는 내용을 빠르게 찾아 쓸 수 있습니다.

4 장. 나는 파이썬을 얼마나 알아야 할까?

이 질문에 대한 답은 명확합니다. **'따라 할 수 있을 정도'면 충분합니다.** 이 가이드와 노트북 시리즈는 변수, 조건문, 반복문, 함수 같은 파이썬 기초 문법을 이미 알고 있다는 전제로 만들어지지 않았습니다. 필요한 문법은 예제 코드 안에서 그때그때 등장하고, 각 코드에는 '이 부분이 왜 필요한지' 설명하는 주석이 함께 있습니다.

4-1. 실제로 필요한 최소 지식

- 변수에 값을 저장하고 꺼내 쓸 수 있다 (`x = 10` 같은 코드가 뭘 하는지 이해)
- 리스트/딕셔너리가 대괄호 `[ ]`, 중괄호 `{ }`로 표현된다는 것을 안다
- 함수는 **이름(괄호 안 값)** 형태로 호출한다는 것을 안다 (예: `print('hello')`)
- 이 세 가지 외에는, 모두 노트북을 풀면서 자연스럽게 손에 익게 됩니다.

4-2. 몰라도 되는 것

역설적으로, 다음과 같은 것들은 **몰라도 전혀 상관없습니다.** 클래스와 객체지향 프로그래밍의 깊은 원리, 알고리즘 최적화, 딥러닝의 수학적 유도 과정, 심지어 어떤 함수의 모든 매개변수를 외우는 것까지도요. 시험은 '올바른 뼈대 코드를 알맞은 값으로 채워 넣을 수 있는가'를 보기 때문에, 뼈대 자체를 이 가이드에서 반복해서 보여드립니다.

4-3. 이 가이드가 여러분을 어디까지 데려다 주는가

PART 3(이론 정리)에서는 EDA 부터 딥러닝까지 각 주제의 핵심 개념을 초보자 눈높이로 요약합니다. PART 4(실무 감각)에서는 실제로 코드를 어떻게 조합하는지 템플릿과 실행 화면으로 보여드립니다. 그리고 이 가이드와 함께 제공되는 **39 개의 Jupyter Notebook**에서는 똑같은 개념을 최소 두 번씩(이론+실무, 실무전용) 서로 다른 데이터로 반복 연습하도록 설계되어 있습니다. 이 흐름을 따라오시면, '나는 코딩을 거의 몰랐는데'로 시작해도 시험장에서 스스로 코드를 완성할 수 있는 수준에 도달하게 됩니다.

막막하면 그냥 따라 치세요

처음에는 이해가 100% 되지 않아도 괜찮습니다.

예제 코드를 그대로 따라 쳐보고, TODO 문제를 풀 때 정답을 먼저 슬쩍 본 뒤 '왜 이렇게 풀었는지' 거꾸로 이해해도 됩니다. 손으로 코드를 쳐보는 반복 자체가 가장 빠른 학습법입니다.

PART 2

시작하기 전 준비

환경 설치부터 오픈북 사이트 실전 활용법, 폴더 사용법까지
실제 화면과 함께 준비합니다

5 장. 개발 환경 설치 - 내 컴퓨터에 실습 환경 만들기

실습 환경은 실제 **AICE Associate 시험장 환경과 최대한 비슷하게** 맞추는 것이 중요합니다. 버전이
다르면 함수의 기본값이나 옵션이 달라져 시험장에서 당황할 수 있기 때문입니다. 아래 순서대로
따라오시면 됩니다.

5-1. Python 설치 확인

이미 Python 이 설치되어 있다면 이 단계는 건너뛰어도 됩니다. 터미널(명령 프롬프트)에서 아래
명령으로 확인하세요.

```
python --version
# Python 3.10.x 가 이상적입니다 (AICE 실습 환경 기준)
```

5-2. 가상환경(venv) 만들기

가상환경은 '이 프로젝트만을 위한 독립된 파이썬 상자'입니다. 다른 프로젝트의 라이브러리 버전과
충돌하지 않도록 반드시 만들어서 사용하세요.

```
# 프로젝트 폴더로 이동한 뒤, 가상환경 생성 (이름: .aice)
python -m venv .aice

# 가상환경 활성화
.aice\Scripts\activate      (Windows)
source .aice/bin/activate   (Mac/Linux)

# 활성화되면 터미널 프롬프트 맨 앞에 (.aice) 표시가 나타납니다
```

(.aice) 표시가 안 보인다면?

가상환경이 제대로 활성화되지 않은 것입니다. 터미널을 껐다 켜거나, 실행 정책 오류(Windows
PowerShell 의 경우 **Set-ExecutionPolicy RemoteSigned** 필요)를 확인해보세요.

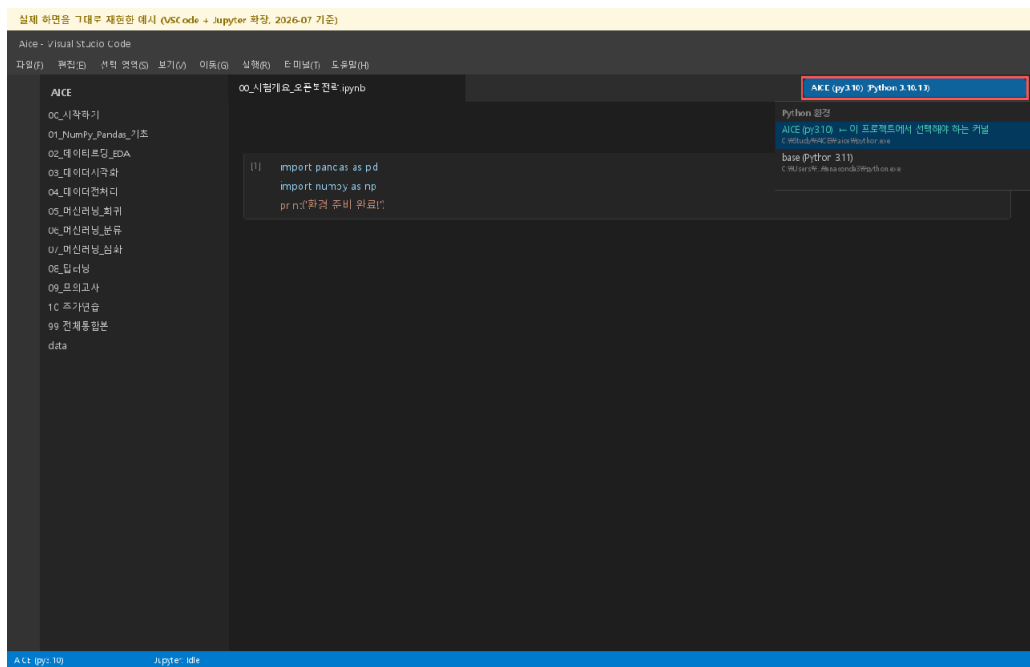
5-3. 라이브러리 설치 (AICE 공식 버전 고정)

아래 버전은 AICE Associate 실습 환경에서 실제로 쓰이는 조합입니다. 버전을 명시해서 설치하면 이 가이드와 연동 노트북의 모든 예제 코드가 그대로 동작합니다.

```
pip install pandas==1.5.3 numpy==1.23.5 matplotlib==3.7.1 seaborn==0.12.1 \
    tensorflow==2.13.1 scikit-learn==1.2.2 xgboost lightgbm imbalanced-learn openpyxl
```

5-4. VSCode + Jupyter 확장 설치

- VSCode(Visual Studio Code)를 설치합니다 (code.visualstudio.com).
- 왼쪽 사이드바의 확장(Extensions) 메뉴에서 '**Python**'과 '**Jupyter**' 확장을 검색해 설치합니다.
- 이 프로젝트 폴더를 VSCode 로 엽니다 (파일 → 폴더 열기).
- 노트북(.ipynb) 파일을 열면 우측 상단에 '커널 선택' 버튼이 보입니다 - 방금 만든 **.aice** 가상환경을 선택하세요.



커널을 못 찾겠다면

우측 상단 버튼을 클릭했을 때 목록에 **AICE (py3.10)**가 안 보이면, 목록 맨 위의 'Python 환경 선택...' → '인터프리터 경로 입력'을 눌러 **.aice\Scripts\python.exe** 경로를 직접 지정하면 됩니다. 커널이 올바르게 선택되면 화면 하단 상태 표시줄에도 커널 이름이 함께 표시됩니다.

5-5. 설치 확인하기

아래 코드를 새 노트북 셀에 실행해서 모든 라이브러리가 정상적으로 불러와지는지 확인하세요.

```
import pandas as pd
import numpy as np
import matplotlib
import seaborn as sns
import sklearn
import tensorflow as tf

print(f'pandas: {pd.__version__}')
print(f'numpy: {np.__version__}')
print(f'scikit-learn: {sklearn.__version__}')
print(f'tensorflow: {tf.__version__}')
print('모든 라이브러리 정상 로드 완료!')
```

6 장. 프로젝트 폴더 구조 사용법

이 프로젝트는 실제 학원 강의처럼 **주제별 번호** 폴더로 구성되어 있습니다. 번호 순서대로 폴더를 열어서 안에 있는 노트북을 처음부터 끝까지 풀면 됩니다.

AICE Associate 학습 폴더 구조

00_시작하기	시험 개요 · 오픈북 전략 · 환경설정
01_NumPy_Pandas_기초	배열/데이터프레임 대량 실전문제
02_데이터로딩_EDA	csv/json/excel 로딩 · 데이터 탐색
03_데이터시각화	countplot-histplot-heatmap 등
04_데이터전처리	결측치·이상치·인코딩·스케일링
05_머신러닝_회귀	회귀 모델 5종 + 평가지표
06_머신러닝_분류	이진/다중분류 모델 + 평가지표
07_머신러닝_심화	교차검증·GridSearch·Pipeline
08_딥러닝	Keras 회귀/이진/다중분류
09_모의고사	10종 실전 모의고사 (90분)
10_추가연습	다른 데이터셋 추가 연습 3종
99_전체통합본	00~08 전체를 하나로 이어붙인 파일

data/ — Titanic·Wine·Iris·California Housing 등 실습용 데이터 14종

각 폴더 사용법

폴더	무엇을 하나요	형식
00_시작하기	시험 개요, 오픈북 전략, 환경설정, 에러 해결가이드, 치트시트, 자동채점기	이론 위주
01_데이터로딩_EDA	라이브러리 불러오기(alias), csv/json/excel 불러오기, 데이터 기본 탐색	이론+실무 · 실무전용
02_NumPy_Pandas_기초	배열/데이터프레임 조작 대량 반복 연습	이론+실무 · 실무전용
03_데이터시각화	그래프별 사용법과 해석 연습	이론+실무 · 실무전용
04_데이터전처리	결측치·이상치·인코딩·스케일링·분할	이론+실무 · 실무전용
05_머신러닝_회귀	회귀 모델 5종 + 평가지표 + 시각화	이론+실무 · 실무전용
06_머신러닝_분류	이진분류·다중분류 모델 + 평가지표	이론+실무 · 실무전용
07_머신러닝_심화	교차검증·GridSearch·Pipeline (심화)	이론+실무 · 실무전용
08_딥러닝	Keras 로 회귀/이진분류/다중분류 구현	이론+실무 · 실무전용
09_모의고사	10 가지 다른 데이터로 만든 실전 모의고사	TODO 중심(정답 포함)
10_추가연습	다른 데이터셋(회귀·다중분류·불균형)에 개념 응용	이론+실무 · 실무전용
99_전체통합본	00~08 을 하나로 이어붙인 파일 (총복습용)	전체
data	모든 노트북이 공통으로 사용하는 실습 데이터	csv/json/xlsx

'이론+실무'와 '실무전용', 두 노트북의 차이

01~08 번과 10 번 폴더에는 노트북이 두 개씩 들어 있습니다. 하나는 이론 설명이 포함된 '이론+실무' 노트북이고, 다른 하나는 같은 주제를 **다른 데이터·다른 문제로 다시** 연습하는 '실무전용'(파일명에 _실무전용이 붙음) 노트북입니다. 이론+실무로 개념을 먼저 익힌 뒤, 실무전용으로 바로 이어서 손을 더 풀면 개념이 확실히 잡힙니다.

각 노트북 안의 구성 패턴

모든 학습 노트북은 아래와 같은 동일한 패턴으로 구성되어 있어, 한 번 익숙해지면 어떤 노트북을 펼쳐도 바로 공부할 수 있습니다.

구성 요소	역할
이론 설명	왜 이 개념이 필요한지, 시험에서 어떻게 나오는지 설명
핵심 개념 정리	함수/문법을 표로 요약
예제 코드	주석으로 '왜 이렇게 쓰는지' 설명하는 실무형 코드
TODO: 실전 문제	시험 문제 형식의 빈 코드 셀 - 직접 풀어보기
정답 보기 (접힘)	클릭해야 펼쳐지는 정답 - 먼저 풀고 나중에 확인

10_추가연습 폴더가 다른 폴더와 헷갈릴 수 있어 부연하면, **09_모의고사**는 이미 배운 내용을 실전처럼 종합해서 푸는 '시험지'이고, **10_추가연습**은 05~08 번에서 배운 개념을 **완전히 다른 도메인 데이터**(캘리포니아 집값, 아이리스 품종, 고객 이탈)에 다시 적용해보며 '응용력'을 기르는 폴더입니다. 폴더 안 README.md 에도 이 차이를 자세히 설명해두었습니다.

PART 3

이론 완전 정리

EDA 부터 딥러닝까지, 초보자 눈높이로 핵심만 요약합니다

연동 노트북 01~08 번과 함께 보면 가장 효과적입니다

7 장. 데이터 분석의 기초 - EDA

EDA(Exploratory Data Analysis, 탐색적 데이터 분석)는 본격적인 전처리나 모델링에 들어가기 전, 데이터의 '첫인상'을 파악하는 단계입니다. 요리하기 전에 냉장고 속 재료가 무엇인지, 상태가 어떤지 먼저 확인하는 것과 같습니다.

7-1. 라이브러리 불러오기 - 시험의 시작

AICE Associate 의 1~2 번 문항은 거의 항상 '라이브러리를 정확한 별칭으로 불러오세요' 형태로 나옵니다. 채점기는 문자 그대로 **pd, np** 같은 alias 를 확인하므로, 문제에 적힌 이름을 그대로 써야 합니다.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

7-2. 다양한 포맷 불러오기

함수	설명
pd.read_csv(path, encoding=, usecols=)	CSV 로딩 (한글 CSV 는 encoding='cp949' 필요할 수 있음)
pd.read_json(path)	JSON 로딩
pd.read_excel(path)	Excel 로딩 (openpyxl 필요)
pd.merge(a, b, on=, how=)	키 기준 조인 (how: inner/left/right/outer)
pd.concat([a, b])	단순 이어붙이기

7-3. 데이터 첫인상 파악하기

본격적인 전처리에 들어가기 전에는 항상 데이터의 '뼈대'를 먼저 확인합니다. **info()**로 컬럼별 타입과 결측치 유무를, **describe()**로 수치형 통계를 확인하는 습관을 들이면 이후 전처리 단계로 매끄럽게 넘어갈 수 있습니다.

함수	설명
df.info()	타입, 결측치, 메모리 사용량
df.describe()	수치형 컬럼의 평균·표준편차·사분위수
df.isnull().sum()	컬럼별 결측치 개수
df.select_dtypes(include='number'/'object')	타입별 컬럼만 추출
df.corr(numeric_only=True)	수치형 변수 간 상관관계수
pd.crosstab(a, b)	두 범주형 변수의 교차표

왜 시험에 나오는가

value_counts()/corr()/nunique() 같은 함수는 EDA 단계에서 가장 먼저 실행하는 명령어들이라, '가장 많이 나타나는 값', '가장 상관관계가 높은 변수'를 묻는 초반 문항에 거의 항상 등장합니다.

7 장 확인 퀴즈

Q1. 결측치 개수를 컬럼별로 확인하는 코드는?

A1. `df.isnull().sum()`

Q2. 두 범주형 변수의 관계를 교차표로 보고 싶을 때 쓰는 함수는?

A2. `pd.crosstab(a, b)`

8 장. 데이터 전처리 완전 정리

실제 응시 후기를 보면 90 분 중 **절반 이상의 시간**이 전처리 단계에서 소요됩니다. 전처리를 빠르고 정확하게 끝내는 능력이 곧 시험 전체 시간 관리와 직결됩니다.

8-1. 결측치 처리

결측치는 남겨두면 이후 모델 학습 단계에서 **에러가 나기 때문에 반드시 처리**해야 합니다. 수치형은 평균/중앙값으로, 범주형은 최빈값으로 채우는 것이 기본이며, 결측치 비율이 지나치게 높은 컬럼은 아예 삭제하는 것도 방법입니다. 특히 조심할 것은 **숨은 결측치**입니다 - 빈 문자열(' ')이나 특수 표기가 실제로는 결측치인데 isnull()로 잡히지 않는 경우가 실무·시험 모두에서 흔합니다.

코드	설명
df['Age'].fillna(df['Age'].median())	중앙값 대체 (이상치 영향이 적음)
df['Embarked'].fillna(df['Embarked'].mode()[0])	최빈값 대체 (범주형에 사용, mode()는 Series 반환)
df.dropna()	결측치가 하나라도 있는 행을 통째로 삭제
df['col'].replace(' ', np.nan).astype(float)	숨은 결측치(공백 문자열)를 NaN 으로 변환 후 형변환
df.groupby('Pclass')['Age'].transform('median')	그룹별 중앙값으로 더 정교하게 대체

8-2. 이상치 탐지 - IQR 과 Z-score

IQR(사분위 범위) 방식은 1 사분위(Q1)와 3 사분위(Q3)의 차이(IQR)의 1.5 배를 벗어나는 값을 이상치로 봅니다. **Z-score** 방식은 (값-평균)/표준편차가 특정 기준(보통 ± 1.96 또는 ± 3)을 넘는 값을 이상치로 봅니다. 이상치는 삭제할 수도, 상한/하한값으로 눌러(clip) 완화할 수도 있습니다.

```
q1, q3 = df['Fare'].quantile([0.25, 0.75])
iqr = q3 - q1
upper = q3 + 1.5 * iqr
df['Fare'] = df['Fare'].clip(upper=upper) # 삭제 대신 상한으로 눌러 데이터 손실 방지
```

8-3. 범주형 인코딩 - 3 가지 방법 비교

방법	코드	특징
LabelEncoder / cat.codes	df['Sex'].astype('category').cat.codes	순서 없이 0,1,2... 부여. 트리 모델엔 무난, 선형모델엔 부적절할 수 있음
get_dummies (원-핫)	pd.get_dummies(df, columns=[...], drop_first=True)	가장 많이 쓰임. drop_first=True 로 다중공선성 방지
OneHotEncoder	OneHotEncoder().fit_transform(df[['Sex']])	sklearn 파이프라인에 넣을 때 유용, categories_로 매핑 확인 가능

8-4. 스케일링 - 3 가지 방법 비교

방법	특징
StandardScaler	평균 0, 표준편차 1 로 변환. 가장 널리 쓰이는 표준 스케일러
MinMaxScaler	값을 정확히 0~1 사이로 압축. 딥러닝 입력에 자주 사용
RobustScaler	중앙값과 IQR 사용. 이상치가 많은 데이터에 안전

데이터 누수(Data Leakage) 주의

스케일러는 반드시 **train 데이터로만 fit** 하고, test 데이터에는 **transform** 만 적용해야 합니다. test 에도 fit 을 다시 하면 '아직 보지 않은 미래 정보'가 학습에 새어 들어가는 데이터 누수가 발생해, 실제보다 성능이 부풀려진 착시가 생깁니다.

8-5. 데이터 분할

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
# stratify=y : 분류 문제에서 클래스 비율을 train/test 에 동일하게 유지
# random_state : 실행할 때마다 같은 결과가 나오도록 시드 고정
```

8 장 확인 퀴즈

Q1. train 으로 fit 한 스케일러를 test 에 적용할 때 써야 하는 메서드는? (fit_transform 이 아니라?)

A1. transform()

Q2. 다중공선성을 피하기 위해 get_dummies 에 반드시 넣어야 하는 옵션은?

A2. drop_first=True

9 장. 데이터 시각화 완전 정리

그래프를 그리는 것보다 해석하는 것이 시험에서 더 중요합니다. 세 가지 원칙만 기억하면 대부분의 해석 문제를 풀 수 있습니다: ① 상관관계 ≠ 인과관계, ② 정규분포는 평균을 중심으로 좌우 대칭인 종 모양, ③ 그래프를 보기 전 축이 무엇을 의미하는지 항상 먼저 확인.

그래프	언제 쓰나	핵심 함수
countplot	범주형 변수의 빈도 비교	<code>sns.countplot(x=, hue=, data=)</code>
histplot / kdeplot	연속형 변수의 분포·치우침 확인	<code>sns.histplot(x=, kde=, bins=)</code>
scatterplot / jointplot	두 연속형 변수의 관계 확인	<code>sns.jointplot(x=, y=, kind='reg')</code>
heatmap	여러 변수 간 상관관계 한눈에 보기	<code>sns.heatmap(corr, annot=True)</code>
boxplot / violinplot	그룹별 분포와 이상치 비교	<code>sns.boxplot(x=, y=, data=)</code>
버블 차트	점 크기로 세 번째 변수까지 표현	<code>plt.scatter(x, y, s=, c=)</code>

annot=True 를 잊지 마세요

heatmap 을 그릴 때 `annot=True` 를 빼먹으면 색만 보이고 숫자가 안 보입니다. 정확한 값을 요구하는 문제에서는 반드시 켜야 합니다.

9 장 확인 퀴즈

Q1. heatmap 에서 칸 안에 숫자를 표시하려면 어떤 옵션을 켜야 하나?

A1. `annot=True`

Q2. 점 크기로 세 번째 변수를 표현하는 그래프 유형은?

A2. 버블 차트 (`plt.scatter` 의 `s=` 옵션 사용)

10 장. 머신러닝 - 회귀 완전 정리

회귀는 '연속적인 숫자 값'을 예측하는 문제입니다(예: 요금, 집값). scikit-learn 의 거의 모든 모델은 동일한 5 단계로 사용합니다 - 이 뼈대만 외워두면 어떤 모델이 나와도 클래스 이름만 바꿔 끼우면 됩니다.

```
# [1]모델 불러오기 → [2]모델 생성 → [3]학습 → [4]예측 → [5]평가
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
from sklearn.metrics import r2_score, mean_absolute_error

print(r2_score(y_test, pred), mean_absolute_error(y_test, pred))
```

모델	모듈
LinearRegression	sklearn.linear_model
DecisionTreeRegressor	sklearn.tree
RandomForestRegressor / GradientBoostingRegressor	sklearn.ensemble
XGBRegressor	xgboost

평가지표	의미
MSE (평균제곱오차)	오차를 제곱해 평균. 큰 오차에 더 민감(패널티가 큼)
MAE (평균절대오차)	오차 절댓값의 평균. MSE 보다 이상치에 덜 민감
R ² (결정계수)	1 에 가까울수록 좋음. 0 이면 평균만 예측한 수준

10 장 확인 퀴즈

Q1. 이상치에 상대적으로 덜 민감한 회귀 평가지표는 MSE 와 MAE 중 무엇인가?

A1. MAE

Q2. R2 가 0 이라는 것은 어떤 수준으로 예측했다는 뜻인가?

A2. 그냥 평균값으로 예측한 것과 비슷한 수준

11 장. 머신러닝 - 분류(이진/다중) 완전 정리

11-1. 이진분류 vs 다중분류

이진분류는 클래스가 정확히 2 개(생존/사망), **다중분류**는 3 개 이상(품종 A/B/C)일 때를 말합니다. 모델을 만들고 학습시키는 코드(fit, predict)는 완전히 동일하고, **달라지는 것은 평가 단계**뿐입니다.

구분	이진분류	다중분류
출력 확률	predict_proba(X[:, 1]) 하나만	클래스별 확률 벡터 전체 필요
precision/recall/f1	옵션 없이 바로 계산	average='macro'/'weighted' 필수 지정
confusion_matrix	2x2	NxN (N=클래스 수)

11-2. 배깅 vs 부스팅

배깅(Bagging)은 중복을 허용해 여러 번 샘플링한 데이터로 여러 모델을 독립적으로 학습시키고 다수결/평균을 냅니다(RandomForest 대표). **부스팅(Boosting)**은 모델을 순차적으로 학습시키며 이전 모델이 틀린 부분에 가중치를 더 줍니다(XGBoost, LGBM, GradientBoosting 대표). 부스팅이 성능은 좋은 경우가 많지만 병렬화가 어렵고 과적합에 더 민감합니다.

11-3. 평가지표 완전 정리

지표	의미	중요한 상황
accuracy	전체 중 맞춘 비율	클래스가 균형일 때
precision	양성이라 예측한 것 중 실제 양성 비율	오탐(false positive)을 줄여야 할 때
recall	실제 양성 중 맞춘 비율	놓치면 안 될 때(질병 진단, 이탈 예측)
f1-score	precision-recall 의 조화평균	두 지표의 균형이 필요할 때
roc_auc	임계값 무관 전반적 분류력	전체적인 분류 성능 비교

11-4. 불균형 데이터 대응

소수 클래스가 5% 미만인 경우도 흔합니다. 이때 모델이 '전부 다수 클래스로 찍어도' accuracy 가 높게 나오는 함정에 빠지기 쉬워, **accuracy 보다 recall/f1/roc_auc 를 봐야 합니다**. 대응법은 class_weight='balanced'(소수 클래스에 더 큰 패널티)와 **SMOTE**(소수 클래스를 합성해 늘리는 오버샘플링) 두 가지가 대표적입니다.

11 장 확인 퀴즈

Q1. 놓치면 안 되는 상황(질병 진단 등)에서 특히 중요한 평가지표는?

A1. recall(재현율)

Q2. 다중분류에서 f1_score 를 계산할 때 반드시 지정해야 하는 옵션은?

A2. average='macro' 또는 'weighted'

12 장. 머신러닝 심화 - 교차검증·튜닝·파이프라인

AICE Associate 핵심 범위 밖에 있지만, 실무에서는 거의 항상 쓰이고 시험에서도 응용문제로 나올 수 있는 개념들입니다.

12-1. 교차검증(K-Fold)

train_test_split 을 한 번만 하면 어쩌다 운 좋게(혹은 나쁘게) 나뉜 데이터로 평가한 결과일 수 있습니다. K-Fold 는 데이터를 K 조각으로 나눠 K 번 서로 다른 조합으로 학습·평가를 반복하고 평균을 내 훨씬 안정적인 성능 추정치를 줍니다.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skf)
print(scores.mean())
```

12-2. 하이퍼파라미터 튜닝

GridSearchCV 는 지정한 값의 **모든 조합**을 교차검증으로 전수 탐색합니다(느리지만 확실함). RandomizedSearchCV 는 일부 조합만 무작위로 탐색해 더 빠릅니다.

```
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    {'n_estimators': [100, 200], 'max_depth': [5, None]}, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

12-3. Pipeline 과 규제 회귀

Pipeline 은 '스케일링 → 모델' 같은 여러 단계를 하나로 묶어 fit/predict 를 한 번에 처리합니다. Ridge(L2)는 가중치를 0 근처로 줄이고, Lasso(L1)는 일부 가중치를 정확히 0 으로 만들어 변수를 자동으로 선택하는 효과가 있습니다.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
pipe = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
pipe.fit(X_train, y_train)
```

12 장 확인 퀴즈

Q1. 여러 하이퍼파라미터 조합을 전수 탐색하는 함수는?

A1. GridSearchCV

Q2. 가중치 일부를 정확히 0 으로 만들어 변수 선택 효과를 주는 회귀 모델은?

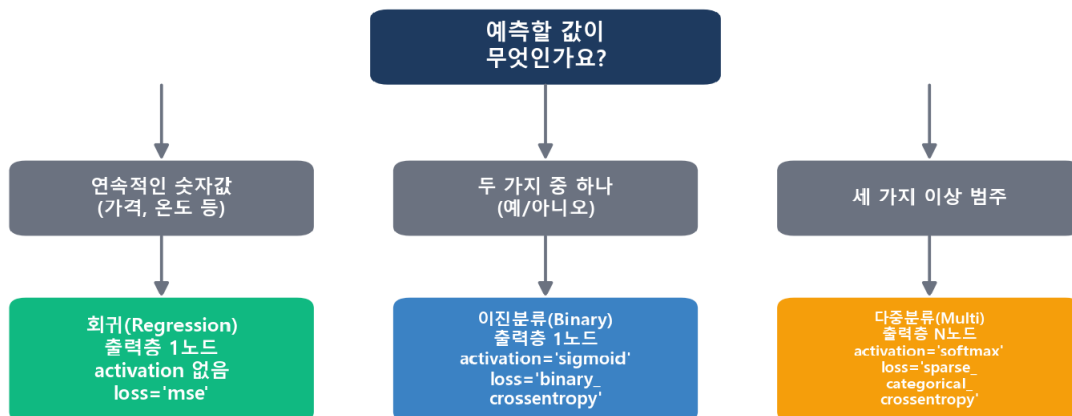
A2. Lasso (L1 규제)

13 장. 딥러닝 완전 정리

AICE Associate 의 딥러닝 문제는 역전파의 수학적 유도 과정을 묻지 않습니다. 대신 '이 문제 유형에 맞게 출력층과 손실함수를 올바르게 설계할 수 있는가'를 봅니다. Sequential API 로 층을 쌓고, compile 로 학습 방식을 정하고, fit 으로 학습시키는 흐름은 어떤 문제든 동일합니다 - 바뀌는 것은 출력층의 노드 수·activation 과 loss 뿐입니다.

문제 유형	출력층 노드 수	activation	loss
회귀	1	없음(항등)	mse
이진분류	1	sigmoid	binary_crossentropy
다중분류	클래스 수	softmax	sparse_categorical_crossentropy

문제 유형 판별 흐름도



머신러닝도 동일한 원리: 회귀는 Regressor 계열, 분류는 Classifier 계열 모델 사용

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

model = Sequential()
model.add(Dense(32, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dropout(0.3))
model.add(Dense(16, activation='relu'))
model.add(Dense(1, activation='sigmoid')) # 이진분류 출력층
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

es = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
history = model.fit(X_train, y_train, epochs=100, batch_size=32,
                    validation_data=(X_test, y_test), callbacks=[es], verbose=0)
```

개념	설명
Dropout	학습 시 일부 노드 연결을 무작위로 꺼서 과적합 방지 (추론 시엔 적용 안 됨)
BatchNormalization	층 출력값을 재정규화해 학습을 안정화·가속
EarlyStopping	검증 손실이 patience epoch 동안 개선 없으면 학습 조기 종료
ModelCheckpoint	가장 좋았던 시점의 모델을 파일로 저장
np.argmax(pred, axis=1)	다중분류의 softmax 확률 벡터를 최종 클래스 라벨로 변환

예측 후처리를 잊지 마세요

이진분류는 predict()가 0~1 확률을 반환하므로 (pred > 0.5).astype(int)로 클래스 변환이 필요합니다. 다중분류는 np.argmax(pred, axis=1)로 변환합니다. 이 후처리 없이 바로 accuracy_score 에 넣으면 에러가 납니다.

13 장 확인 퀴즈

Q1. 다중분류 출력층의 activation 은 무엇인가?

A1. softmax

Q2. 검증 손실이 개선되지 않을 때 학습을 자동으로 멈추는 콜백은?

A2. EarlyStopping

PART 4

실무 감각 익히기

코드 템플릿, 자주 만나는 에러, 그리고 실제 실행 화면으로
이론을 실무 감각으로 연결합니다

14 장. 실전 코드 템플릿 총정리

아래는 이 시리즈 전체에서 배우는 내용의 '완성된 뼈대'입니다. 시험 직전 마지막 점검용으로, 또는 문제를 풀다가 코드가 기억나지 않을 때 바로 펼쳐보는 용도로 활용하세요.

14-1. 결측치 처리

```
print(df.isnull().sum())
df['Age'] = df['Age'].fillna(df['Age'].median())

df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

14-2. 전처리 (인코딩 · 분할 · 스케일링)

```
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)

from sklearn.model_selection import train_test_split
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

14-3. 머신러닝 5 단계

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_s, y_train)
pred = model.predict(X_test_s)
```

```
print(accuracy_score(y_test, pred))
print(f1_score(y_test, pred, average='macro')) # 다중분류는 average 지정
```

14-4. 딥러닝 5 단계

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping

model = Sequential()
model.add(Dense(16, activation='relu', input_shape=(X_train_s.shape[1],)))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
history = model.fit(X_train_s, y_train, epochs=50, batch_size=32,
                    validation_data=(X_test_s, y_test), callbacks=[es])

pred = model.predict(X_test_s)
pred_label = (pred > 0.5).astype(int) # 다중분류는 np.argmax(pred, axis=1)
```

14-5. 고득점 응용 3 종 세트

아래 세 가지는 알아두면 응용 문항에서 크게 유리한 테크닉입니다.

```
# ① 그룹별 통계치로 결측치 정교하게 채우기
df['Age'] = df['Age'].fillna(df.groupby('Pclass')['Age'].transform('median'))

# ② 날짜 파생변수 생성
df['date_col'] = pd.to_datetime(df['date_col'])
df['year'] = df['date_col'].dt.year
df['dayofweek'] = df['date_col'].dt.dayofweek # 0:월요일 ~ 6:일요일

# ③ 학습된 모델 저장 및 불러오기
model.save('best_model.h5')
from tensorflow.keras.models import load_model
loaded = load_model('best_model.h5')
```

15 장. 자주 발생하는 에러와 해결법

시험 중 흔하게 마주치는 에러 메시지와 그 원인, 해결 방법을 정리했습니다.

ValueError: could not convert string to float

원인: 모델 학습(fit)이나 스케일링을 할 때 문자열 컬럼이 포함되어 발생합니다.

```
print(X_train.dtypes) # 원인 컬럼 확인
```

```
df = pd.get_dummies(df, drop_first=True) # 인코딩
```

ValueError: Input contains NaN, infinity or a value too large

원인: 데이터에 결측치(NaN)가 아직 남아있는데 모델 학습을 시도했을 때 발생합니다.

```
print(df.isnull().sum()) # 남은 결측치 확인  
df = df.fillna(df.mean(numeric_only=True))
```

AttributeError: 'Series' object has no attribute 'flatten'

원인: pandas Series 객체에는 numpy 배열용 메서드인 flatten 이 없습니다.

```
y_test.values.flatten() # Series 를 numpy 배열로 바꾼 뒤 적용
```

Shape mismatch (스케일링/인코딩 관련 에러)

원인: Train 과 Test 의 컬럼 수가 다를 때(Test 에 특정 범주가 없어 get_dummies 결과 컬럼 수가 달라짐) 발생합니다.

```
df = pd.get_dummies(df, drop_first=True) # 분할 전에 인코딩 먼저 수행  
X = df.drop(columns=['target']); y = df['target']  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

ValueError: Found array with dim 3. Estimator expected <= 2

원인: scikit-learn 모델에 3 차원 이상의 배열(주로 이미지 데이터)을 넣었을 때 발생합니다.

```
X_train_flat = X_train.reshape(X_train.shape[0], -1) # 2 차원으로 펼치기
```

ConvergenceWarning (LogisticRegression 등)

원인: 반복 학습이 지정된 횟수 안에 수렴하지 못했다는 경고입니다. 에러는 아니지만 성능에 영향을 줄 수 있습니다.

```
LogisticRegression(max_iter=5000) # max_iter 를 넉넉하게 늘려줌
```

UnicodeDecodeError: 'cp949' codec can't decode byte / 'utf-8' codec can't decode byte

원인: CSV 파일이 만들어질 때 쓰인 인코딩과 pd.read_csv()가 기본으로 시도하는 인코딩이 달라서 발생합니다 (한글이 섞인 데이터에서 흔함).

```
df = pd.read_csv('data.csv', encoding='cp949') # utf-8 로 실패하면 cp949(한글 윈도우 기본 인코딩)로 시도  
df = pd.read_csv('data.csv', encoding='utf-8-sig') # 그래도 안 되면 반대로 utf-8-sig 로 시도
```

KeyError: '컬럼명'

원인: 존재하지 않는 컬럼명을 참조했을 때 발생합니다. 오타이거나, get_dummies/drop 등으로 이미 사라진 컬럼일 수 있습니다.

```
print(df.columns.tolist()) # 실제 컬럼명 확인 (대소문자·띄어쓰기 오타 주의)
```

ValueError: Shapes (None, 1) and (None, 3) are incompatible

원인: 다중분류인데 출력층 노드를 1 개로 만들었거나, 반대로 이진분류인데 여러 개로 만들었을 때, 또는 loss 를 문제 유형과 다르게 설정했을 때 발생합니다.

```
model.add(Dense(1, activation='sigmoid')) # 이진분류: 출력 노드 1 개 + sigmoid
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.add(Dense(3, activation='softmax')) # 다중분류(클래스 3 개): 출력 노드 3 개 + softmax
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

16 장. 실제 문제풀이 예시

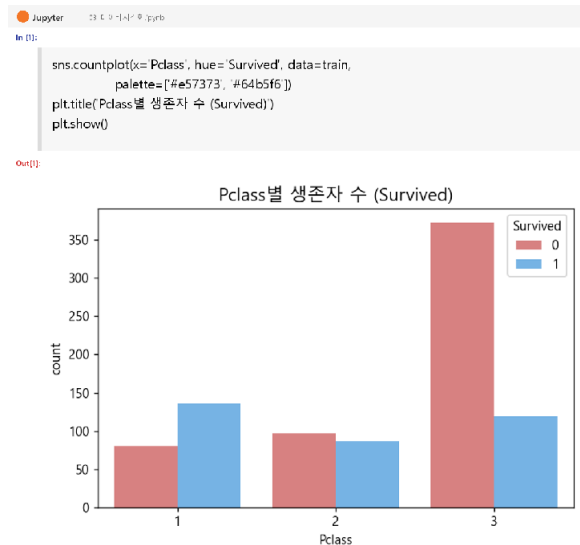
아래 화면은 이 프로젝트의 실습 노트북을 실제로 실행해서 나온 진짜 결과입니다.

코드를 어떻게 조합해 결과를 만들어내는지 순서대로 따라가 보세요.

16-1. 실제 문제풀이 예시 (실행 화면 수록)

예시 1. 좌석 등급(Pclass)별 생존자 시각화

countplot 에 hue 옵션을 주면 범주형 변수를 다시 한 번 나눠 비교할 수 있습니다. 아래 결과에서 3 등석 승객의 사망자 수가 압도적으로 많다는 것을 한눈에 확인할 수 있습니다.



예시 2. RandomForest 이진분류 - confusion matrix 확인

실제 Titanic 데이터로 RandomForestClassifier를 학습시킨 결과입니다. 96명은 사망을 사망으로, 49명은 생존을 생존으로 맞췄고(대각선), 20명은 실제 생존자를 사망으로 잘못 예측(False Negative)했습니다. 이 예시에서 accuracy는 81.0%, f1-score는 74.2%가 나왔습니다.

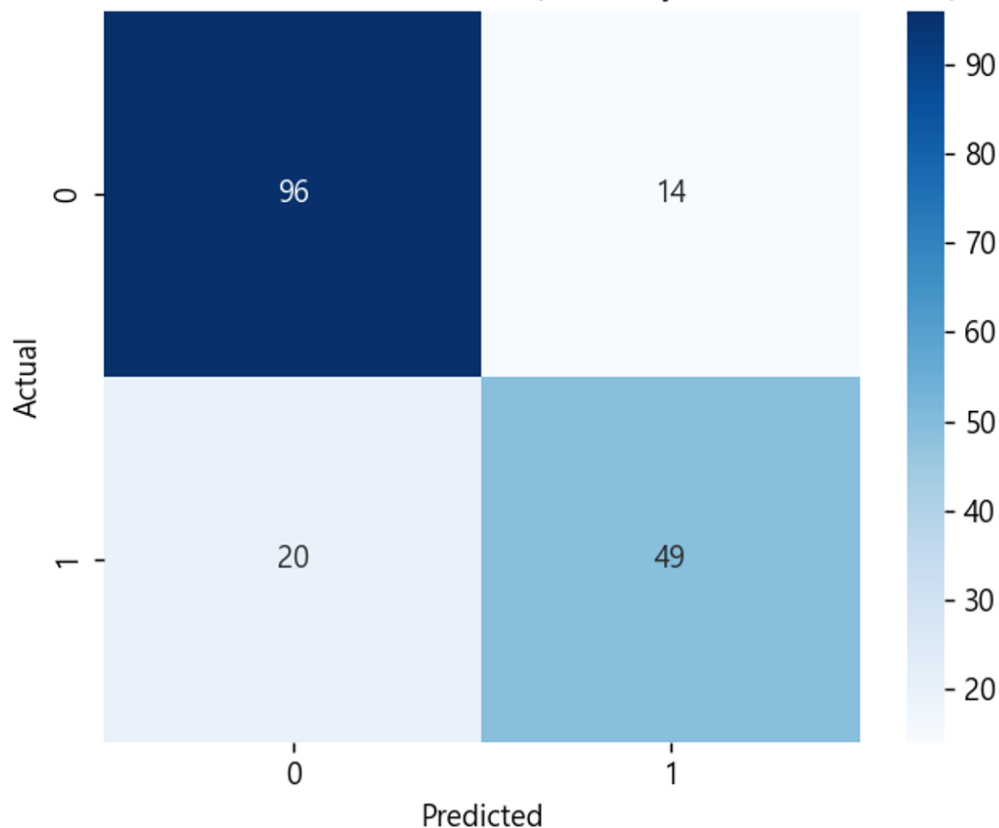
Jupyter 06_머신러닝_분류_이진.ipynb

In [1]:

```
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
pred = rf.predict(X_test)
cm = confusion_matrix(y_test, pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title(f'accuracy={accuracy_score(y_test,pred):.3f}')
plt.show()
```

Out[1]:

RandomForest Confusion Matrix (accuracy=0.810, f1=0.742)



예시 3. 딥러닝 이진분류 - 학습 곡선(Loss Curve) 확인

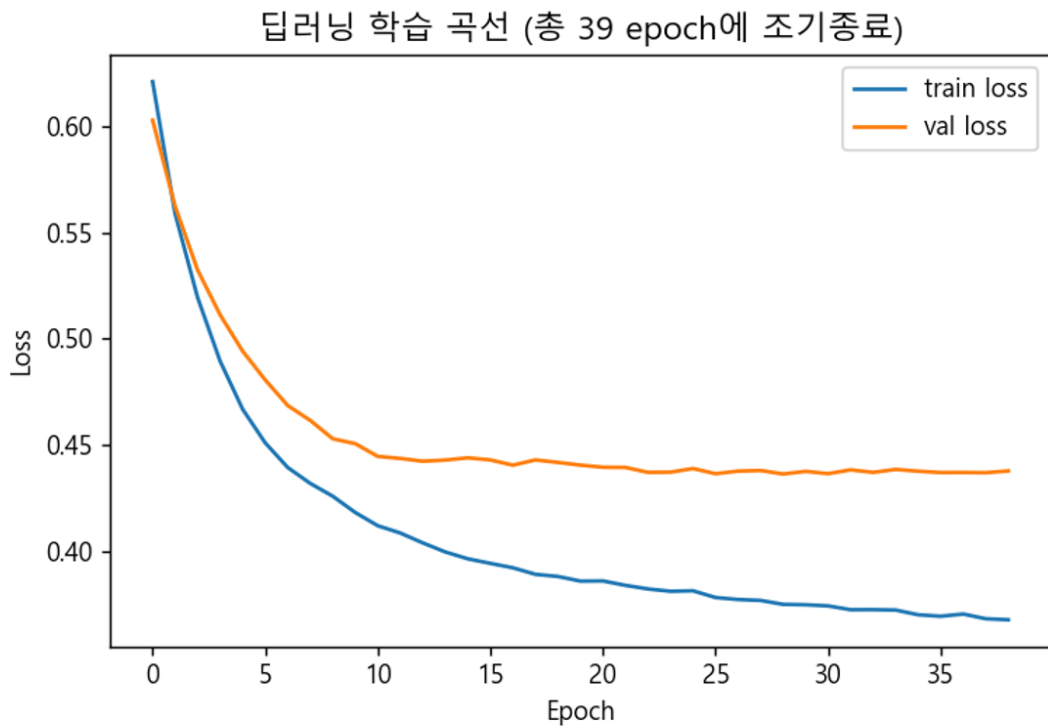
동일한 데이터를 Keras Sequential 모델로 학습시킨 결과입니다. EarlyStopping 이 적용되어 있어, 검증 손실이 더 이상 개선되지 않자 100 epoch 를 다 채우지 않고 자동으로 학습이 조기 종료되었습니다. train loss 와 val loss 가 함께 감소하는 모습에서 정상적으로 학습되고 있음을 확인할 수 있습니다.

Jupyter 09_딥러닝_분류_이진.ipynb

In [1]:

```
history = model.fit(X_train_s, y_train, epochs=100,
                    validation_data=(X_test_s, y_test),
                    callbacks=[es], verbose=0)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.legend(); plt.show()
```

Out[1]:



이 결과를 그대로 재현해보고 싶다면

위 세 화면은 각각 **03_데이터시각화**, **06_머신러닝_분류_이진**, **09_딥러닝_분류_이진** 노트북의 코드로 만들어졌습니다. 연동된 노트북을 열어 직접 실행해보면 완전히 동일한 결과를 눈으로 확인할 수 있습니다.

16-2. 미니 모의고사 실전 시뮬레이션

지금까지는 결과 화면 위주로 봤다면, 이번에는 **09_모의고사** 폴더의 실제 파일(모의고사 01_Titanic_생존자예측.ipynb) 안에 문제가 정확히 어떤 형태로 적혀 있고, 그 문제를 어떤 순서로 사고해서 코드로 옮기는지 처음부터 끝까지 따라가 보겠습니다. 노트북을 열어본 적이 없어도 이 흐름만 알면 바로 감이 잡힐 것입니다.

문제 1. 데이터 첫인상 확인하기 (1 교시: 데이터 로딩 & EDA)

노트북에 실제로 적힌 문제

문제 1. df 의 shape 과 컬럼별 결측치 개수를 출력하세요.

어떻게 접근할까: '결측치 개수'라는 단어가 보이면 반사적으로 `isnull().sum()`을 떠올려야 합니다. `shape` 은 데이터 크기(행, 열)를 튜플로 반환하는 가장 기본적인 확인 명령입니다. 이 문제는 어렵게 생각할 필요 없이, **7 장에서 배운 EDA 기본 명령어를 그대로 호출하면** 끝나는 문제입니다.

 Jupyter 모의고사01_Titanic_생존자예측.ipynb

In [1]:

```
print(df.shape)
print(df.isnull().sum())
```

Out[1]:

```
(891, 12)
Age      177
Cabin    687
Embarked   2
(나머지 컬럼: 결측치 0개)
```

해설: 실행 결과 891 행 12 열의 데이터이고, Age(177 개)·Cabin(687 개)·Embarked(2 개) 컬럼에 결측치가 있다는 것을 바로 확인할 수 있습니다. 이 숫자를 보는 순간 '3 교시(전처리)에서 이 세 컬럼을 처리해야 겠구나'라는 계획이 자연스럽게 세워집니다 - EDA 는 이후 전처리 전략을 세우기 위한 정찰 단계입니다.

문제 4. Pclass 별 생존율 시각화하기 (2 교시: 데이터 시각화)

노트북에 실제로 적힌 문제

문제 4. Pclass 별 생존율을 bar 그래프로 그리세요.

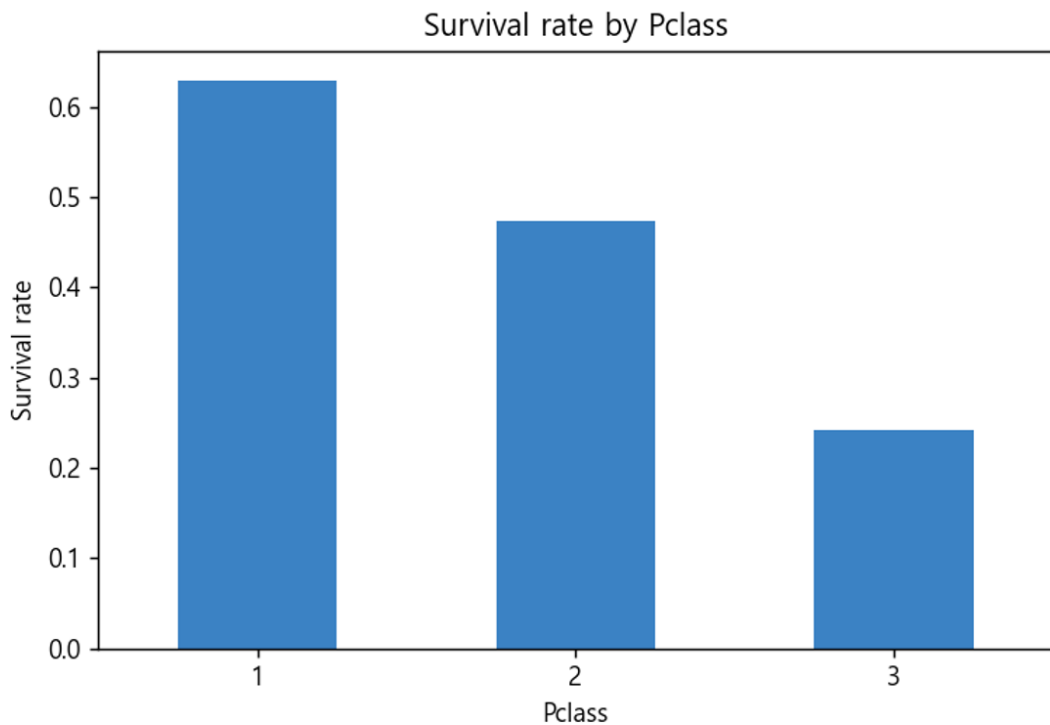
어떻게 접근할까: '~별 ○○율'이라는 표현이 나오면 groupby가 필요하다는 신호입니다. 'Pclass 별 생존율' → Pclass로 묶어서(groupby) Survived의 평균(mean, 0/1의 평균 = 비율)을 구한다는 뜻입니다. 그 결과(Series)에 바로 .plot(kind='bar')를 붙이면 그래프 함수를 따로 부르지 않고도 빠르게 그릴 수 있습니다 - 9장에서 배운 'pandas 내장 시각화'를 활용하는 대표적인 상황입니다.

Jupyter 모의고사01_Titanic_생존자예측.ipynb

In [1]:

```
df.groupby('Pclass')['Survived'].mean().plot(kind='bar', color='#3b82c4')
plt.title('Survival rate by Pclass')
plt.ylabel('Survival rate')
plt.show()
```

Out[1]:



해설: 그래프를 보면 1 등석(Pclass=1)의 생존율이 가장 높고, 3 등석으로 갈수록 낮아지는 것이 뚜렷합니다. 이 관찰 자체가 '좌석 등급이 생존에 영향을 미치는 중요한 변수'라는 근거가 되어, 이후 모델링 단계에서 Pclass를 반드시 포함해야 하는 이유가 됩니다.

문제 11. RandomForest 로 학습·평가하기 (4 교시: 머신러닝 모델링)

노트북에 실제로 적힌 문제

문제 11. RandomForestClassifier(n_estimators=100, random_state=42)를 학습시키고 accuracy, f1-score 를 구하세요.

어떻게 접근할까: 이 문제는 사실 이 가이드의 14 장에서 외운 '머신러닝 5 단계 템플릿'을 그대로 적용하는 문제입니다. 문제 문장에 이미 클래스 이름(RandomForestClassifier)과 하이퍼파라미터(n_estimators=100, random_state=42)가 정확히 명시되어 있으므로, **[1]불러오기 → [2]생성 → [3]학습 → [4]예측 → [5]평가** 순서로 그대로 옮기기만 하면 됩니다. 단, 이 시점에서 3 교시 전처리가 이미 끝나 있어야(결측치 처리, 인코딩, 분할) 이 코드가 에러 없이 돌아갑니다.

 Jupyter 모의고사01_Titanic 생존자예측.ipynb

In [1]:

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(accuracy_score(y_test, pred))
print(f1_score(y_test, pred))
```

Out[1]:

```
0.8156 (accuracy)
0.7519 (f1-score)
```

해설: accuracy 81.6%, f1-score 75.2%가 나왔습니다. 두 지표가 어느 정도 차이가 나는 것은 Titanic 데이터가 완전히 균형 잡힌 데이터는 아니기 때문입니다(생존 38% : 사망 62%). 만약 이 차이가 훨씬 크게 벌어진다면 11 장에서 배운 '불균형 데이터 대응'(class_weight, SMOTE)을 검토해야 한다는 신호로 읽을 수 있습니다.

이 흐름에서 꼭 기억할 것

세 문제 모두 '문제 문장의 핵심 단어 → 배운 개념 → 템플릿 코드'로 이어지는 동일한 사고 과정을 거쳤습니다. 새로운 문제를 만나도 당황하지 말고, 이 가이드의 PART 3 에서 배운 개념 중 어떤 것이 지금 이 문장과 연결되는지부터 찾아보세요. 09_모의고사 폴더의 나머지 9 개 파일로 이 감각을 반복해서 다지는 것이 합격의 지름길입니다.

PART 5

합격 전략

시험 당일 시간을 아끼는 실전 전략과
흔한 실수, 그리고 14 일 학습 로드맵까지

17 장. 오픈북 시험 전략 8 가지 + α

① alias 규칙을 반드시 지키기

`import pandas as pd` 처럼 문제에서 요구한 별칭 그대로 사용해야 채점기가 인식합니다. 1~2 번 문항이 거의 항상 '~를 ~로 불러오세요' 형태로 나오는 이유이기도 합니다.

② 템플릿 코드를 몸매 새기기

결측치 처리, 인코딩, 모델 5 단계 같은 코드는 매번 거의 똑같은 뼈대를 씁니다. 뼈대를 외워두면 시험 중에는 값(컬럼명, 옵션)만 바꿔 끼우는 작업만 남습니다.

③ Shift+Tab 으로 함수 정보 확인하기

함수 이름 뒤에 커서를 두고 Shift+Tab 을 누르면 함수 시그니처, 독스트링, 옵션 기본값을 바로 확인할 수 있습니다. 오픈북 문서를 매번 새 탭에서 찾는 것보다 훨씬 빠릅니다.

④ 변수명이 문제마다 바뀌는 상황에 대비하기

어떤 문제는 `X_train`, 다른 문제는 `x_train` 처럼 대소문자가 바뀌기도 합니다. 문제에 명시된 변수명을 정확히 그대로 복사해서 쓰는 습관을 들이세요.

⑤ 전처리에 시간의 절반 이상이 걸린다는 것을 각오하기

실제 응시 후기를 보면 90 분 중 전처리 파트에서만 절반 이상의 시간이 소요됩니다. 전처리를 빨리 끝내는 연습이 곧 시험 전체 시간 관리입니다.

⑥ 임시저장을 자주 활용하기

시스템 오류나 실수로 인한 손실을 막기 위해 문제를 풀 때마다 임시저장하는 습관을 들이세요.

⑦ 막히면 일단 넘어가고 두 번째 회독하기

템플릿을 알고 있다면 시험 시간에 문제를 2 번 정도 풀어볼 여유가 생깁니다. 막히는 문제에 시간을 너무 쓰지 말고, 아는 문제부터 빠르게 채운 뒤 돌아오세요.

⑧ 데이터 타입/구조를 항상 먼저 확인하기

.info(), .head()를 습관적으로 먼저 찍어보면 결측치·타입 문제를 코드를 짜기 전에 미리 발견할 수 있습니다.

+α. 예측 후처리를 빼먹지 않기

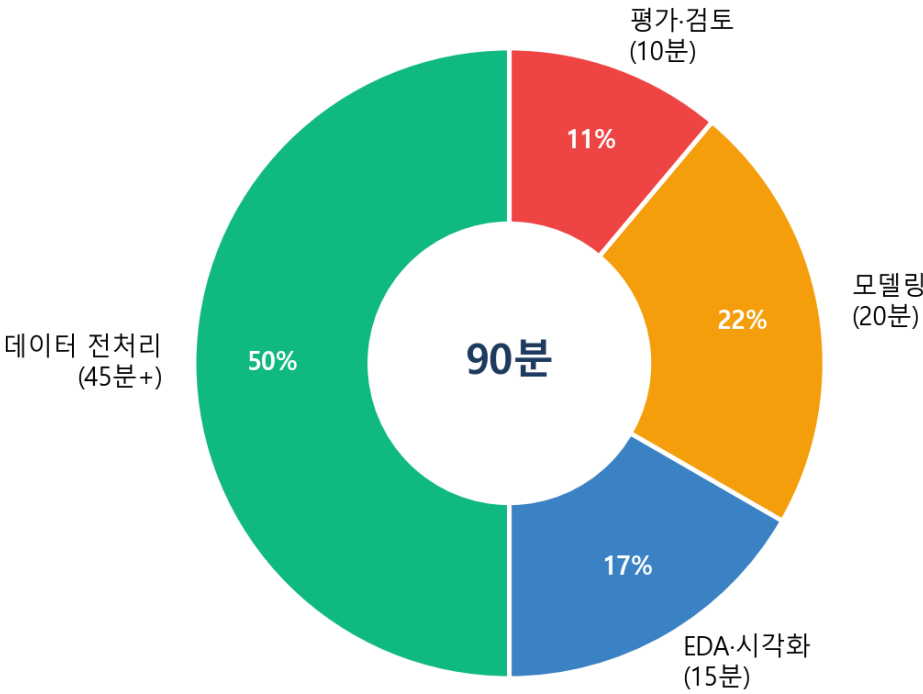
이진분류는 0.5 임계값 변환, 다중분류는 argmax 변환을 잊으면 평가 함수 호출 시 에러가 납니다. 모델링의 마지막 단계로 항상 체크하는 습관을 들이세요.

18 장. 시간 배분 전략 (90 분 분배)

90 분이라는 시간은 넉넉해 보이지만, 문제를 하나씩 읽고 이해하는 시간까지 고려하면 결코 여유롭지 않습니다. 아래는 권장하는 시간 배분 가이드입니다.

구간	권장 시간	내용
1 단계: 문제 훑어보기	5 분	전체 문항을 빠르게 훑으며 난이도와 유형을 파악
2 단계: 데이터 로딩 & EDA	10 분	라이브러리 불러오기, 데이터 확인
3 단계: 전처리	35 분	결측치·이상치·인코딩·스케일링·분할 (가장 시간이 많이 걸리는 구간)
4 단계: 모델링 & 평가	30 분	머신러닝/딥러닝 모델 학습, 평가지표 계산
5 단계: 검토 & 재확인	10 분	빠뜨린 문항 확인, alias·변수명 재확인, 제출

실전 시간 배분 감각 (예시)



시간이 부족하다고 느껴진다면

전처리 템플릿(14 장)을 완전히 외워둘수록 3 단계 시간이 크게 줄어듭니다. 전처리는 어떤 데이터가 나와도 패턴이 거의 동일하므로, 이 부분을 자동화된 습관으로 만드는 것이 시간 관리의 핵심입니다.

19 장. 자주 하는 실수 Top 10

#	실수	설명
1	alias 를 문제와 다르게 사용	채점기는 정확한 이름을 요구합니다. import pandas as pd 를 as pandas 로 쓰면 오답 처리될 수 있습니다.
2	train 으로 fit 한 스케일러를 test 에 다시 fit	test 는 반드시 transform()만 적용해야 데이터 누수를 막을 수 있습니다.
3	불균형 데이터에서 accuracy 만 확인	소수 클래스를 무시해도 accuracy 가 높게 나올 수 있어, recall/f1 도 함께 봐야 합니다.
4	이진/다중분류 출력층 activation 혼동	이진분류는 sigmoid, 다중분류는 softmax 입니다. 반대로 쓰면 학습 자체가 이상하게 진행됩니다.
5	숨은 결측치(공백 문자열 ' ') 놓침	isnull()로 안 잡히는 결측치가 있습니다. .info()로 타입이 이상한 컬럼(object 인데 숫자여야 하는 컬럼)을 먼저 확인하세요.
6	get_dummies 를 train/test 분할 후 따로 적용	Test 데이터에 없는 범주가 있으면 컬럼 수가 달라져 Shape mismatch 에러가 납니다. 인코딩을 분할 전에 먼저 하세요.
7	average 옵션 없이 다중분류 지표 계산	precision_score/recall_score/f1_score 는 다중분류에서 average 를 반드시 지정해야 계산됩니다.
8	random_state 를 지정하지 않음	실행할 때마다 점수가 미세하게 달라져 재현이 안 됩니다. 습관적으로 random_state=42 를 붙이세요.
9	reset_index(drop=True)를 빠뜨림	분할·필터링 후 인덱스가 뒤섞인 채로 이후 연산을 하면 예상과 다른 결과가 나올 수 있습니다.
10	예측 확률을 그대로 accuracy_score 에 대입	딥러닝의 predict() 결과(확률)를 임계값 변환 없이 그대로 넣으면 에러가 납니다. 0.5 기준 변환(이진) 또는 argmax(다중)를 거쳐야 합니다.

20 장. 14 일 학습 로드맵

아래 로드맵은 하루 1~2 시간 학습을 기준으로 설계되었습니다. 이미 관련 지식이 있다면 더 빠르게, 처음이라면 각 구간을 이틀씩 늘려서 진행해도 좋습니다.

일차	학습 내용
1 일차	PART 1~2 정독 + 환경설정 + 00_시작하기 폴더
2~4 일차	01_데이터로딩_EDA ~ 03_데이터시각화 (이론+실무, 실무전용 모두)
5~7 일차	04_데이터전처리 ~ 06_머신러닝_분류
8~11 일차	07_머신러닝_심화 ~ 10_추가연습
12~14 일차	09_모의고사 반복 풀이 + 99_전체통합본으로 총복습

14일 완성 학습 로드맵 (예시 - 본인 속도에 맞게 조절)

1일차	환경설정 + 00_시작하기
2~3일차	01_Numpy_Pandas 기초
4일차	02_EDA + 03_시각화
5일차	04_데이터전처리
6~7일차	05~06_머신러닝(회귀/분류)
8일차	07_머신러닝_심화
9~10일차	08_딥러닝(회귀/이진/다중)
11일차	10_추가연습 (다른 데이터셋)
12~13일차	09_모의고사 10종 반복
14일차	99_전체통합본 총복습

12~14 일차의 모의고사 반복이 특히 중요합니다. 이론을 다 안다고 느껴져도, 실제 시험처럼 시간을 재고 처음부터 끝까지 풀어보는 경험이 있어야 시험장에서 당황하지 않습니다.

"완벽하게 다 이해하지 못해도 괜찮습니다. 반복하면 됩니다."

마치며 - AICE Associate 합격, 이제 시작입니다

여기까지 읽으셨다면, AICE Associate 가 요구하는 것이 생각보다 명확하고 손에 잡히는 범위라는 것을 느끼셨을 것입니다. 이 가이드에서 다룬 이론 요약과 전략은 **출발선**일 뿐, 진짜 실력은 연동된 39 개의 노트북을 직접 풀어보면서 만들어집니다.

처음 노트북을 열었을 때 낯설고 막막하게 느껴지더라도, 하나씩 풀어나가다 보면 어느 순간 '이 패턴, 어디서 본 것 같은데'라는 느낌이 들기 시작할 것입니다.

그 순간이 바로 여러분이 이 시험을 준비를 잘 하고 있다는 뜻이고 데이터를 다루는 감각이 체화되는 신호라고 보시면 됩니다.

이 가이드와 노트북 시리즈를 끝까지 따라오신 모든 분들의 합격을 진심으로 응원합니다.

부록 A. 문제유형별 모델·모듈·평가함수 총정리표

문제유형	모델명	모듈	평가함수
회귀	LinearRegression	sklearn.linear_model	mse, mae, r2_score
회귀	DecisionTreeRegressor	sklearn.tree	"
회귀	RandomForestRegressor / GradientBoostingRegressor	sklearn.ensemble	"
회귀	XGBRegressor / LGBMRegressor / ExtraTreesRegressor	xgboost / lightgbm / sklearn.ensemble	"
회귀	VotingRegressor (여러 모델 앙상블)	sklearn.ensemble	"
이진/다중분류	LogisticRegression / DecisionTreeClassifier	sklearn.linear_model / tree	accuracy, precision, recall, f1, roc_auc
이진/다중분류	RandomForestClassifier / ExtraTreesClassifier / GradientBoostingClassifier	sklearn.ensemble	"
이진/다중분류	XGBClassifier / LGBMClassifier	xgboost / lightgbm	"
이진/다중분류	SGDClassifier / KNeighborsClassifier / SVC	sklearn.linear_model / neighbors / svm	"
이진/다중분류	VotingClassifier (여러 모델 앙상블)	sklearn.ensemble	"

부록 B. 폴더/노트북 전체 목록

폴더	이론+실무 노트북 / 실무전용 노트북
00_시작하기	00_시작하기.ipynb (단일 통합 파일: 진단테스트 · 시험개요 · 오픈북전략 · 치트시트 · 에러가이드 · 자동채점기)
01_데이터로딩_EDA	01_EDA_데이터로딩_탐색 / _실무전용
02_NumPy_Pandas_기초	02_NumPy_Pandas_기초 / _실무전용
03_데이터시각화	03_데이터시각화 / _실무전용
04_데이터전처리	04_데이터전처리 / _실무전용
05_머신러닝_회귀	05_머신러닝_회귀 / _실무전용
06_머신러닝_분류	06_머신러닝_분류_이진, 06_머신러닝_분류_다중 / 각 _실무전용
07_머신러닝_심화	07_머신러닝_심화_교차검증_튜닝 / _실무전용
08_딥러닝	08_딥러닝_회귀, 08_딥러닝_분류_이진, 08_딥러닝_분류_다중 / 각 _실무전용
09_모의고사	모의고사 01~10 (10 개 파일, TODO+정답 포함)
10_추가연습	EX01_회귀, EX02_다중분류, EX03_불균형데이터 / 각 _실무전용
99_전체통합본	00~08 번을 한 파일로 이어붙인 총복습용 노트북

부록 C. 추가 실전 예시 (실행 화면 수록)

16 장에 이어, 변수 중요도 해석 예시를 하나 더 보여드립니다. 이 역시 실제 데이터로 직접 실행해서 나온 결과입니다.

예시 4. Feature Importance 로 중요 변수 해석하기

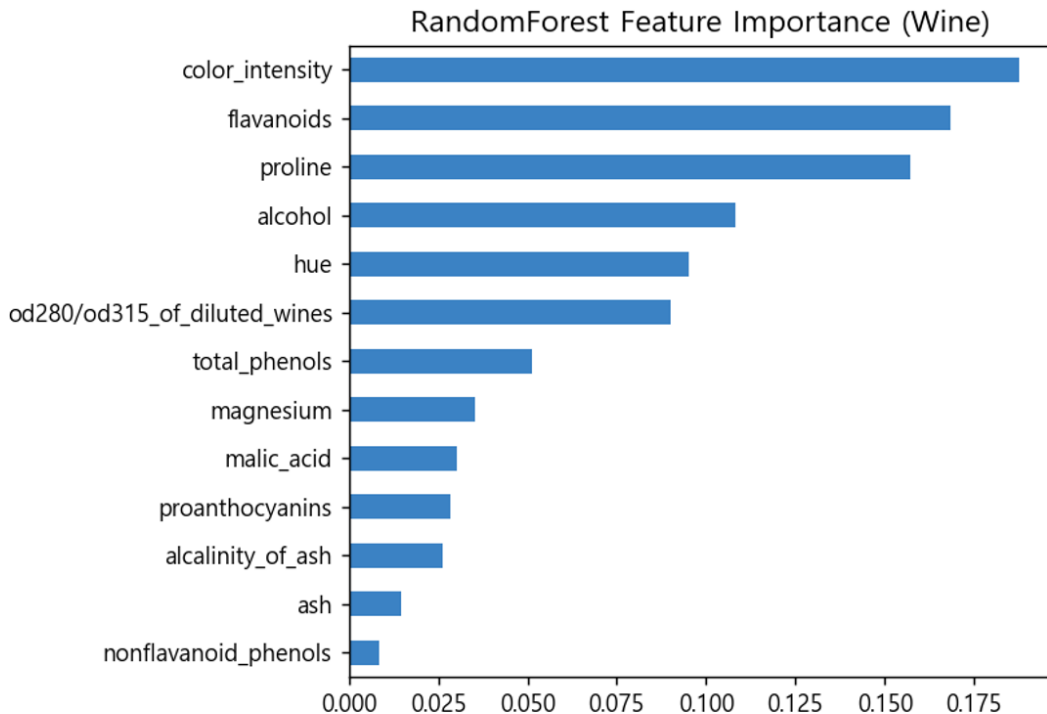
트리 기반 모델(RandomForest, XGBoost 등)은 학습 후 `feature_importances_` 속성으로 각 변수가 예측에 얼마나 기여했는지 확인할 수 있습니다. '가장 중요한 변수가 무엇인가'를 묻는 문제에서 바로 활용하는テクニック입니다. `sort_values()`로 정렬한 뒤 `barh`로 그리면 어떤 변수가 가장 큰 막대를 갖는지 한눈에 보입니다.

Jupyter 05_머신러닝_회귀.ipynb

In [1]:

```
imp = pd.Series(rf.feature_importances_, index=X.columns)
imp.sort_values().plot(kind='barh')
plt.title('RandomForest Feature Importance')
plt.show()
```

Out[1]:



부록 D. 핵심 용어 사전

낮선 용어가 나올 때마다 찾아볼 수 있도록 시험에 자주 등장하는 용어를 정리했습니다.

용어	설명
EDA (탐색적 데이터 분석)	본격적인 분석 전 데이터의 구조·타입·결측치·분포를 먼저 살펴보는 단계
결측치 (Missing Value)	값이 비어 있는 데이터. fillna/dropna 로 처리
이상치 (Outlier)	다른 값들과 동떨어진 극단값. IQR-Z-score 로 탐지
인코딩 (Encoding)	문자열(범주형) 데이터를 숫자로 변환하는 과정
원-핫 인코딩 (One-Hot Encoding)	범주마다 0/1 로 이루어진 별도 컬럼을 만드는 인코딩 방식
스케일링 (Scaling)	변수마다 다른 값의 범위를 비슷하게 맞추는 과정
데이터 누수 (Data Leakage)	테스트(미래) 정보가 학습 과정에 새어 들어가 성능이 부풀려지는 현상
과적합 (Overfitting)	훈련 데이터에만 지나치게 맞춰져 새로운 데이터에는 성능이 떨어지는 현상
교차검증 (Cross Validation)	데이터를 여러 조각으로 나눠 반복 학습·평가해 안정적인 성능을 추정하는 방법
하이퍼파라미터 (Hyperparameter)	모델 학습 전에 사람이 직접 지정하는 설정값 (n_estimators, max_depth 등)
배깅 (Bagging)	데이터를 중복 허용해 샘플링한 뒤 여러 모델을 독립적으로 학습, 다수결/평균을 내는 방식
부스팅 (Boosting)	이전 모델의 오차를 보완하며 모델을 순차적으로 학습시키는 방식
앙상블 (Ensemble)	여러 모델의 예측을 결합해 더 안정적인 성능을 얻는 기법
혼동행렬 (Confusion Matrix)	실제값과 예측값의 조합별 개수를 표로 나타낸 것
정밀도 (Precision)	양성으로 예측한 것 중 실제 양성인 비율
재현율 (Recall)	실제 양성 중 모델이 맞춘 비율
불균형 데이터 (Imbalanced Data)	클래스별 데이터 개수 차이가 큰 데이터
SMOTE	소수 클래스 데이터를 합성해 늘리는 오버샘플링 기법
활성화 함수 (Activation Function)	신경망 층의 출력을 비선형으로 변환하는 함수 (relu, sigmoid, softmax 등)
손실함수 (Loss Function)	모델의 예측이 실제값과 얼마나 다른지 측정하는 함수, 학습의 기준이 됨
에폭 (Epoch)	전체 학습 데이터를 한 번 다 학습하는 단위
배치 크기 (Batch Size)	한 번의 가중치 업데이트에 사용하는 데이터 개수
조기 종료 (Early Stopping)	검증 성능이 더 이상 개선되지 않으면 학습을 자동으로 멈추는 콜백
드롭아웃 (Dropout)	학습 시 일부 노드를 무작위로 꺼서 과적합을 방지하는 기법
파이프라인 (Pipeline)	전처리와 모델을 하나로 묶어 순서대로 실행하는 구조

부록 E. 자주 묻는 질문 (FAQ)

Q. 파이썬을 하나도 모르는데 며칠 만에 준비할 수 있나요?

A. 개인차가 있지만, 이 가이드의 14 일 로드맵을 하루 2~4 시간씩 따라가면 충분히 가능합니다. 중요한 것은 문법을 외우는 것이 아니라 템플릿 코드를 손에 익히는 반복입니다.

Q. 이론을 다 이해하지 못하면 문제를 못 푸나요?

A. 아닙니다. 시험은 '빠대 코드를 알맞게 채워 넣을 수 있는가'를 봅니다. 이 가이드의 템플릿과 노트북의 예제를 반복해서 따라 치는 것만으로도 상당수 문항을 풀 수 있습니다.

Q. 오픈북인데 왜 이렇게 어렵게 느껴지나요?

A. 2025 년부터 '자유 검색'이 아니라 '지정 문서 7 종만 열람'으로 바뀌었기 때문입니다. 기초 개념과 함수 이름을 미리 알고 있어야 문서에서 원하는 내용을 빠르게 찾을 수 있습니다.

Q. 회귀·분류·딥러닝 중 어디에 시간을 더 써야 하나요?

A. 전처리(8 장)에 가장 많은 시간이 걸리므로 이 부분을 확실히 자동화된 습관으로 만드는 것이 우선입니다. 모델링 자체는 5 단계 템플릿이 거의 동일해 상대적으로 빠르게 체득됩니다.

Q. 노트북을 다 풀었는데도 불안합니다. 어떻게 해야 하나요?

A. 09_모의고사 폴더의 10 개 파일을 시간을 재고 실전처럼 반복해서 풀어보세요. 아는 내용도 시간 압박 속에서 풀어보는 경험이 있어야 실전 감각이 붙습니다.

Q. 딥러닝 이론(역전파, 경사하강법 수식)을 깊게 공부해야 하나요?

A. 아닙니다. AICE Associate 는 수학적 유도 과정을 묻지 않습니다. 출력층 설계(activation, loss 선택)와 Keras API 사용법에 집중하세요.

Q. 에러가 나면 오픈북 문서에서 어떻게 찾아야 하나요?

A. 에러 메시지 자체보다, 어떤 함수를 쓰다가 에러가 났는지를 먼저 파악한 뒤 해당 함수를 오픈북 사이트에서 검색하세요. 15 장의 자주 발생하는 에러 목록도 함께 참고하세요.

Q. 실무전용 노트북과 모의고사, 뭐가 다른가요?

A. 실무전용은 이론+실무 노트북과 같은 주제를 다른 데이터로 다시 연습하는 '반복 학습'용이고, 모의고사는 여러 주제를 섞어 실전처럼 종합적으로 푸는 '실전 시뮬레이션'용입니다.

Q. XGBoost 와 LightGBM 중 어떤 걸 먼저 익혀야 하나요?

A. 코드 사용법(5 단계 템플릿)은 거의 동일하므로 먼저 XGBoost 로 흐름을 익힌 뒤, LGBMClassifier 도 같은 방식으로 바로 적용해보면 됩니다. Windows 에서는 lightgbm 을 pandas 보다 먼저 import 해야 하는 DLL 충돌 이슈가 있다는 점만 기억하세요.

Q. 시험 중에 코드가 실행이 안 되면 어떻게 하나요?

A. 당황하지 말고 바로 위 셀부터 순서대로 다시 실행됐는지 확인하세요. 변수가 이전 셀에서 정의되지 않았거나, 셀 실행 순서가 꼬여 있는 경우가 대부분입니다.

부록 F. 학습 체크리스트

시험 전 스스로 점검해보는 최종 체크리스트입니다. 인쇄해서 하나씩 표시해보세요.

No.	체크 항목
1	라이브러리를 정확한 alias 로 불러올 수 있다 (pd, np, plt, sns, tf)
2	결측치를 평균/중앙값/최빈값으로 각각 채우는 코드를 바로 쓸 수 있다
3	숨은 결측치(공백 문자열 등)를 찾아 처리할 수 있다
4	IQR/Z-score 로 이상치를 탐지하고 clip/제거할 수 있다
5	get_dummies 와 LabelEncoder 의 차이를 설명할 수 있다
6	StandardScaler 를 train 에만 fit 하고 test 는 transform 만 하는 이유를 설명할 수 있다
7	train_test_split 에서 stratify=y 가 왜 필요한지 설명할 수 있다
8	회귀 모델 5 단계 템플릿을 코드 없이도 순서대로 말할 수 있다
9	이진분류와 다중분류의 평가지표 차이(average 옵션)를 설명할 수 있다
10	confusion_matrix 를 heatmap 으로 그리고 해석할 수 있다
11	불균형 데이터에서 accuracy 대신 어떤 지표를 봐야 하는지 안다
12	회귀/이진분류/다중분류의 출력층 activation 과 loss 조합을 안다
13	EarlyStopping 과 Dropout 이 각각 무엇을 방지하는지 설명할 수 있다
14	모의고사 10 개 중 최소 3 개 이상을 시간을 재고 풀어봤다

부록 G. 딥러닝 모듈 경로 총정리

Keras 는 클래스가 `tf.keras` 아래 여러 하위 모듈에 나뉘어 있어 처음에는 어디서 무엇을 import 해야 할지 헷갈리기 쉽습니다. 자주 쓰는 클래스의 정확한 import 경로를 정리했습니다.

클래스	import 경로	역할
Sequential	<code>tensorflow.keras.models</code>	층을 순서대로 쌓는 기본 모델 구조
Dense	<code>tensorflow.keras.layers</code>	완전연결(fully-connected) 레이어
Dropout	<code>tensorflow.keras.layers</code>	과적합 방지용 드롭아웃 레이어
BatchNormalization	<code>tensorflow.keras.layers</code>	층 출력값 재정규화로 학습 안정화
EarlyStopping	<code>tensorflow.keras.callbacks</code>	검증 손실 정체 시 학습 조기 종료
ModelCheckpoint	<code>tensorflow.keras.callbacks</code>	가장 좋은 시점의 모델을 파일로 저장
<code>load_model</code>	<code>tensorflow.keras.models</code>	저장된 모델(.h5 등) 불러오기
Adam	<code>tensorflow.keras.optimizers</code>	가장 널리 쓰이는 옵티마이저(문자열 'adam'으로도 사용 가능)

레이어와 콜백을 헷갈리지 마세요

Dense/Dropout/BatchNormalization 처럼 **모델 구조를 이루는 것은 layers** 모듈에,

EarlyStopping/ModelCheckpoint 처럼 **학습 과정을 제어하는 것은 callbacks** 모듈에 있습니다. 이 구분만 기억해도 tensorflow 공식 문서에서 헤매는 시간이 크게 줄어듭니다.

부록 H. 찾아보기 - 핵심 함수 색인

자주 찾게 되는 함수를 가나다/알파벳 순으로 정리했습니다.

어느 장에서 다뤘는지 함께 표시했습니다.

함수	장	함수	장
classification_report	11 장	clip()	8 장
confusion_matrix	11 장	cross_val_score	12 장
Dense / Dropout	13 장	dropna() / fillna()	8 장
EarlyStopping	13 장	f1_score	11 장
get_dummies	8 장	GridSearchCV	12 장
groupby()	7 장	heatmap	9 장
info() / describe()	7 장	LabelEncoder	8 장
np.argmax	13 장	Pipeline / make_pipeline	12 장
Ridge / Lasso	12 장	r2_score	10 장
RandomForestClassifier/Regressor	10~11 장	recall_score / precision_score	11 장
roc_auc_score	11 장	Sequential	13 장
SMOTE	11 장	StandardScaler	8 장
train_test_split	8 장	value_counts()	7 장
XGBClassifier / XGBRegressor	10~11 장		